HOW A COMPUTER WAKES UP

The picture guide to OS development - plus a complete coding course
so you can BUILD one on your Mac



Based on “The Little Book About OS Development”
by Erik Helin & Adam Renberg (2015)

Zero coding knowledge needed - just curiosity!

? What's inside this mini-book

PART ONE - THE BIG PICTURE

[1] **What even IS an operating system?** — *the boss inside your computer*
[2] **How a computer wakes up** — *power button - BIOS - GRUB - your OS*
[3] **The secret handshake** — *hello, 0xCAFEBABE!*
[4] **Talking to the screen** — *a giant board of light-up tiles*
[5] **Keeping a diary** — *the serial port logbook*
[6] **Dividing up the big room** — *segmentation*
[7] **Ding-dong! Interrupts** — *how the keyboard gets attention*
[8] **Two uniforms** — *boss mode vs visitor mode*
[9] **Fake addresses** — *paging & virtual memory*
[10] **The landlord** — *handing out memory rooms*
[11] **Unpacking a program** — *loading and running code*
[12] **Filing cabinets** — *file systems*
[13] **Ringing the service bell** — *system calls*
[14] **Doing everything at once** — *multitasking*
[15] **You made it!** — *the whole journey in one breath*

PART TWO - LEARN TO CODE, FROM ZERO

[16] **What programming really is** — *recipes for a very obedient helper*
[17] **Your first C programs** — *values, variables, choices and loops*
[18] **Functions** — *recipes inside recipes*
[19] **Memory, pointers and structs** — *the mailboxes behind it all*
[20] **Compiler, linker and Make** — *from text file to running program*
[21] **Assembly in one sitting** — *the CPU's own language*
[22] **Set up your Mac workshop** — *tools, folders and make run*

PART THREE - BUILD YOUR OS, MILESTONE BY MILESTONE

[23] **Milestone 0: the spark** — *loader.s, link.ld and QEMU*
[24] **Milestone 1: your C code runs** — *kmain and the magic flags*
[25] **Milestone 2: the screen driver** — *letters on the light board*

[26] **Milestone 3: the logbook** — *the serial port driver*
[27] **Milestone 4: the boss's uniform** — *the GDT, for real*
[28] **Milestone 5: doorbells** — *IDT, PIC and exceptions*
[29] **Milestone 6: your OS reads keys** — *the keyboard driver*
[30] **Milestone 7: fake addresses on** — *paging*
[31] **Milestone 8: the landlord** — *the page-frame allocator*
[32] **Milestone 9: user mode!** — *system calls and your first app*
[33] **Your finished OS** — *debugging cookbook & next steps*

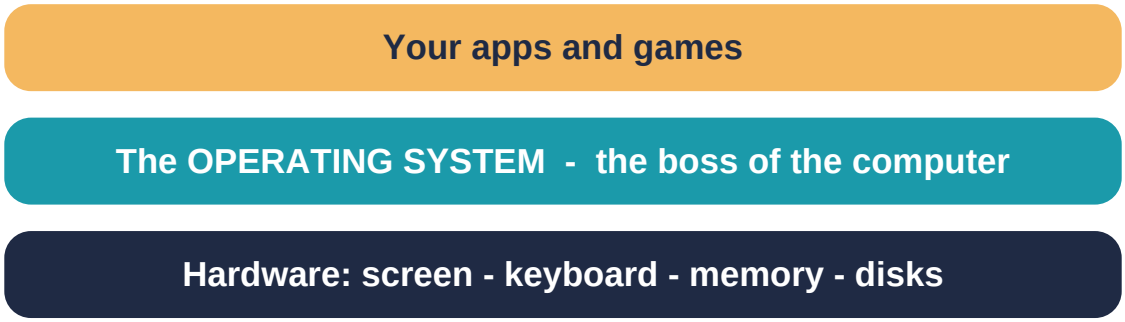
How to read this

Each chapter is short. The pictures carry the ideas, and the words use everyday language - relay races, landlords, doorbells and filing cabinets. Whenever the real book uses a scary word, we translate it here.

1 What even IS an operating system?

Imagine a brand-new computer straight from the factory. It has a brain (the processor), a big desk for papers (memory), a screen, a keyboard... but it just sits there. It has no idea how to show you anything or listen to you.

An **operating system** (OS) is the **boss program** that moves in and runs the whole place. It decides who gets to use the desk space, it draws letters on the screen, it listens to the keyboard, it keeps your files tidy, and it makes sure programs don't fight.



The book builds that middle layer from scratch - the boss itself!

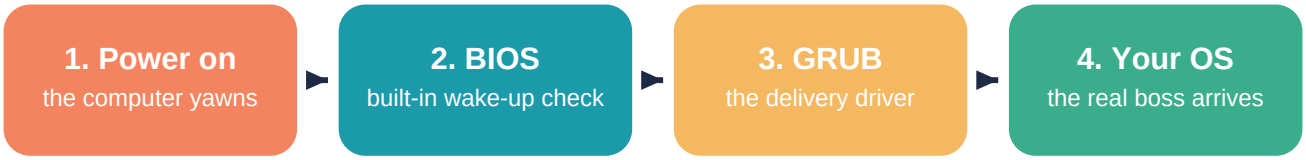
In plain words

The processor is the worker. Memory is the desk. The screen and keyboard are the doors and windows. The operating system is the manager who knows how everything fits together.

The little book you read teaches you to BUILD that boss, starting from absolutely nothing. Not by memorizing theory - by actually making it, one tiny step at a time. Its best tip: after every small step, test it. Then, if something breaks, you know exactly which step broke it.

2 How a computer wakes up

Pressing the power button does not magically start your OS. It is more like a relay race - a chain of tiny programs, each one waking up the next, bigger one:



Each one hands the baton to the next, until the operating system takes over.

Just like a relay race at school sports day!

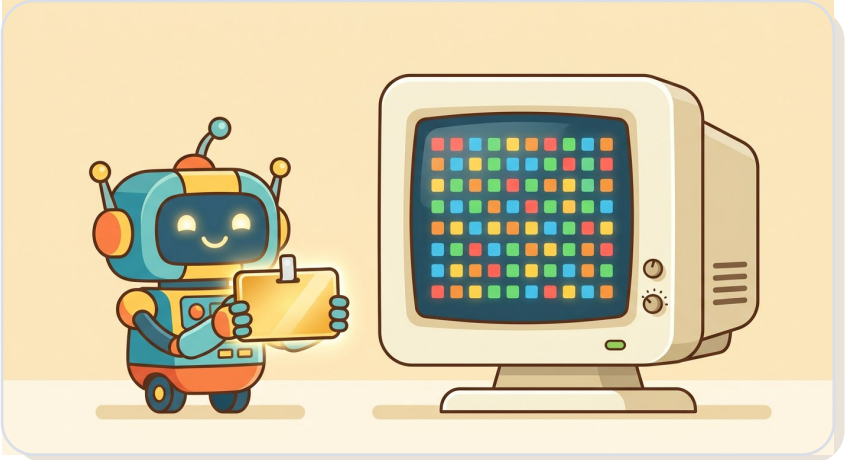
BIOS is the first runner. It is built into the hardware itself, like a wake-up alarm glued inside the machine. It stretches a little (“is the keyboard there? the screen? OK!”) and then hands over.

The **bootloader** is the second runner. Its whole life purpose is to find an operating system and get it onto the stage. Writing one is fiddly work, so the book uses a famous existing one: **GRUB**. Think of GRUB as a delivery driver who already knows where the boss's office is - he carries your OS into memory (the big desk) and says: “Go!”

Fun fact

The very first runner must squeeze into a tiny 512-byte mailbox on the disk - about the size of one short text message! That is why nobody writes bootloaders just for fun.

3 The secret handshake: Hello, 0xCAFEBABE!



Before GRUB starts your code, it checks the secret password at the door.

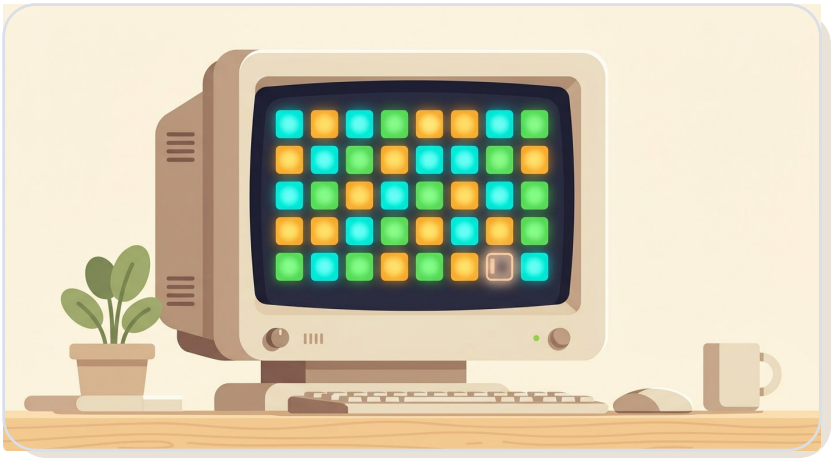
GRUB is careful. Before it jumps to your code, it looks for a **magic number** - a secret password written at the very top of your program. The password is **0x1BADB002** (hex-speak for a memorable handshake), and the tradition is to answer with a second funny number when you succeed: **0xCAFEBABE**. Yes, really - it reads like “cafe babe”. Programmers have a sense of humor too.

The tiniest possible OS does just one thing: it writes 0xCAFEBABE into the CPU's hand and waves forever. The CPU's “hands” are called **registers** - tiny name-tag slots where it holds the number it is working with right now.

How do we know it worked?

The pretend computer (a simulator called Bochs) can print out all its registers when you stop it. If you see CAFEBABE sitting in one of them - your code ran! That is the “Hello, world!” of operating systems. Smallest OS ever: it does absolutely nothing, and everyone is thrilled.

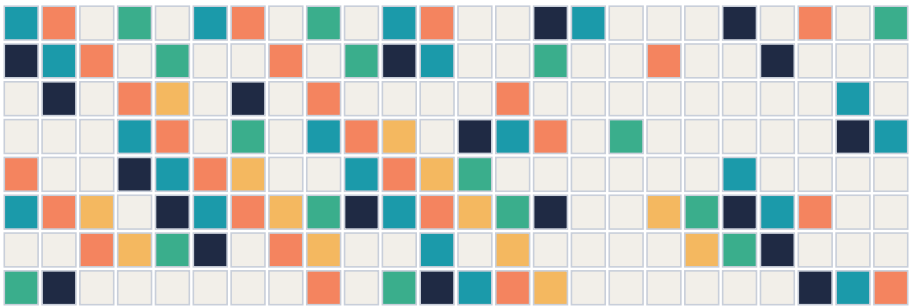
4 Talking to the screen



The text screen works like a board of light-up tiles.

How does a letter appear on the screen? The text screen is really just a **giant light board**: 80 tiles across, 25 tiles down - 2,000 tiles in total. Each tile holds one letter plus its colors, packed into 2 tiny bytes (a byte = 8 tiny on/off switches).

The screen = a light board of 80 x 25 tiles (here: a tiny sample)



Each tile holds: one letter + its colors, packed into 2 tiny bytes

Write to memory spot 0xB8000 - and the tiles light up!

Here is the sneaky trick: the screen is wired to a fixed spot in memory - spot number **0xB8000** (that is just its locker number). Write a letter's code there... and it instantly lights up on the real screen! Writing to memory IS talking to the screen.

The blinking cursor is shyer - it is not driven by memory but through a side door called an **I/O port** (two little numbered chutes, 0x3D4 and 0x3D5). Your code slips a note through the chute: “cursor, please move to tile 80”.

What is a “driver”?

Instead of every part of your OS whispering hardware numbers, you write one helpful middleman - a DRIVER. The rest of the system just says “print this sentence, please” and the driver handles the nerdy details. Same idea as a TV remote: one friendly button, complicated magic inside.

5 Keeping a diary: the serial port

When things go wrong, it really helps to know what your OS was doing. Enter the **serial port** - an old-fashioned plug that speaks in a steady stream of bytes, like a telegraph line or a walkie-talkie channel.

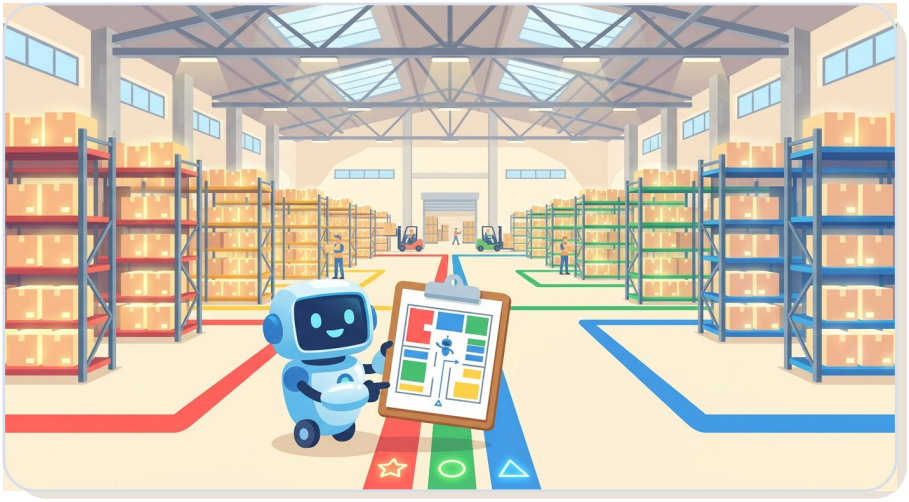
Neat trick: the pretend computer Bochs can be told to “write down everything said on that line into a file”. Boom - a **logbook**! Or like an airplane's black box: whatever your OS did, it leaves a trail you can read afterwards.

Before chatting, agree on how

Two radios must tune to the same channel before they can talk. The serial port needs setup too: how FAST we speak (the speed), how we double-check for mistakes (parity & stop bits), and how big the in-tray is (buffers). The book walks through each dial - then builds a driver with a nice DEBUG / INFO / ERROR log function, just like real software.

With a screen driver AND a diary driver, the OS can finally say where it is and what it sees. From here on, whenever we hit a bug, the diary will save us hours of guessing.

6 Dividing up the big room: segmentation



Memory: one giant warehouse - segmentation paints it into zones.

Memory is one gigantic warehouse of tiny lockers, all numbered. Without rules, anything could scribble anywhere - one grumpy program could erase everyone else's work.

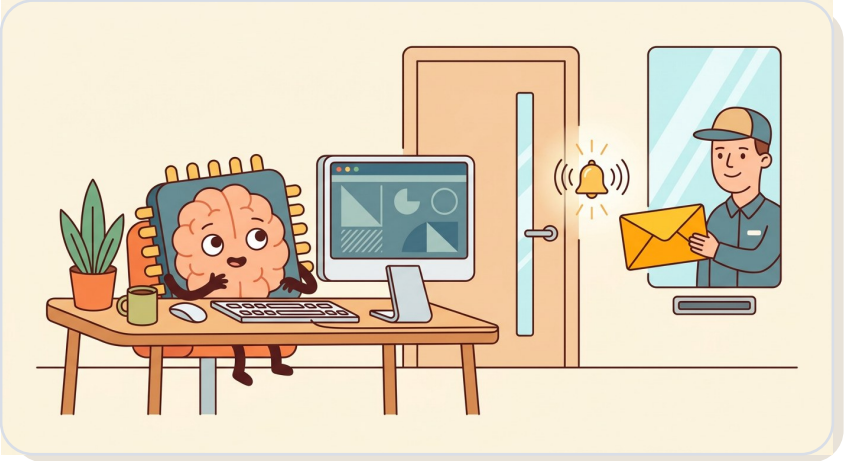
Segmentation is like painting the warehouse floor into colored zones: “program code lives here”, “data lives there”, “boss-only area”. Each zone has a starting point, a size limit, and a permission level. The CPU carries six little **name tags** (called segment registers) that say which zone it is currently allowed into.

Boss level 0, visitor level 3

x86 chips have four “privilege levels” numbered 0 to 3, where 0 is the supreme boss and 3 is the visitor badge. The OS builds its zones at level 0. Later, programs run at level 3 and simply CANNOT barge into boss areas - the hardware itself blocks the door.

Honestly? Modern systems barely use segmentation - everyone switched to the next trick (paging, coming soon). But you still need a minimal setup with the right name tags, so we do exactly that: two zones (code + data), just enough to get the privilege levels working.

7 Ding-dong! Interrupts & the keyboard



An interrupt is a doorbell: “Stop a second - the keyboard needs you!”

The CPU is a busy worker that could compute forever. But you just pressed a key! How does anyone get its attention? With a **doorbell**. In computer speak: an **interrupt**.

When a key is pressed (or a timer rings, or a wire has news), the hardware goes DING. The CPU pauses, looks up “who handles this doorbell?” in a big phone book (the **IDT** - Interrupt Descriptor Table), runs that helper routine, then returns to its own work *exactly* where it left off - mid-thought, registers and all.

The mailroom (PIC) and the keyboard's secret code

A little sorter chip called the **PIC** decides whose ring gets through (you don't want doorbells from broken devices you cannot answer). And here is a surprise: the keyboard does NOT send letters - it sends **scan codes**, like Morse tap-patterns. Your OS must translate tap #30 into “a”. Also: always say “thanks, got it!” back to the PIC, or it will not ring again!

8 Two uniforms: boss mode vs visitor mode



The castle keeps the master keys inside - visitors stay in the village.

Remember the privilege levels? Now they matter. **Kernel mode** is the manager's uniform with the master key - the OS may touch anything. **User mode** is the visitor badge - apps get a fenced play area and nothing else.

Why bother? Because apps will crash, freeze and misbehave - and when a visitor trips, the whole building must NOT fall over. In user mode, a broken app just... quietly fails, while the boss stays safe and sound.

The road ahead

To run a program in visitor mode, the book first lets one run in BOSS mode (easiest), then builds the full gate: special zones (chapter 6), fake addresses (chapter 9), and a proper switch through the CPU's “return from interruption” door - the only way to put on the visitor uniform.

9 Fake addresses: paging & virtual memory



Every tenant thinks the whole building is theirs - the manager knows better.

Here is the big magic trick of modern computers. Every program is told: “The whole address space is yours - lockers 0 to 4 billion!” Obviously there is not that much real memory. It is a polite lie.

The trick is called **paging**. Memory is chopped into equal **pages** - boxes of 4,096 bytes each. A big **catalog** (the page table) translates every fake address the program uses into the real box where the data actually lives. A tiny hardware clerk inside the CPU (the MMU) does this translation billions of times per second.

Every program thinks it owns the whole building - the catalog quietly redirects.



Knock on a locker that was never rented? BEEP! That is a "page fault".

Why fake addresses are wonderful

1) Each app gets its own privacy fence - it cannot peek at others. 2) Memory can be scattered anywhere, yet the app still sees a nice tidy hallway. 3) The boss can mark pages “read-only” or “employees only”. 4) Apps can think they own more memory than the machine even HAS.

10 The landlord: handing out memory rooms

With fake addresses flowing, somebody must track the REAL boxes: which are free, which are taken, and which hold the elevator machinery and must never be rented out. That somebody is the **page frame allocator** - the landlord of memory.

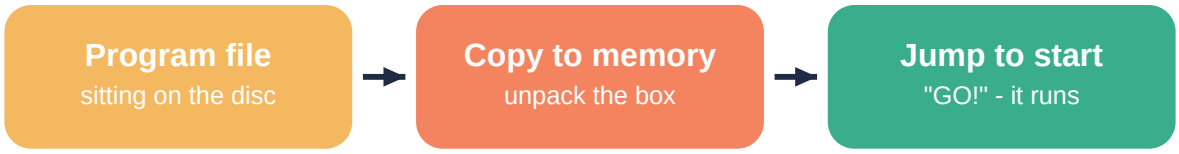
Good news: GRUB already snooped the machine at startup and hands your OS a map of the building - which floors exist, which are RAM, which are special equipment. The book's advice: use a simple **checklist** (one checkbox per box - a “bitmap”) and remember to paint the kernel's own rooms as TAKEN. (GRUB does NOT do that for you. Oops.)

And the supply closet: a heap

With a landlord in place, the OS can have its own “supply closet” (a kernel HEAP): whenever it needs a desk, a list or a name tag, it asks the closet (malloc / free in the book). The closet asks the landlord for more rooms whenever it runs out.

One clever plumbing trick from the book: what if the OS needs a new catalog page to map a box, but mapping THAT needs a catalog page...? The fix: keep one “emergency window” slot permanently free, used only to peek at a fresh box long enough to file its paperwork. Every OS has a trick like this up its sleeve.

11 Unpacking a program



The book uses GRUB "modules": the delivery driver drops the package in memory before the OS even starts. Easy peasy!

How does an app get onto the stage? A real OS reads programs off disks using drivers and file systems - lots of work. The book takes a shortcut: GRUB can already carry “packages” (called **modules**) into memory before the OS starts. The OS just finds the package address and hops to it.

The first test program is hilariously small: it writes **0xDEADBEEF** into a register and loops forever (another classic programmer joke). Run it, stop the simulator, check the log: BEEF found? Then a real, separate program ran inside your OS!

Real apps, later

Later, real apps are written in C but packed into a “flat” shape starting with a tiny 3-step “welcome mat” at the very front: call main, get the result, loop forever. A tiny linker script orders the pieces (the full how-to is in the book's chapter 11).

12 Filing cabinets: file systems



A file system is the cabinet, the labels, and the librarian rules.

Why do computers need file systems? Ask: which program starts first? Where do settings live? How does an app find its data? In the Unix world, the answer to nearly everything is: “it is a file”. The file system is the **filing cabinet** that makes that possible.

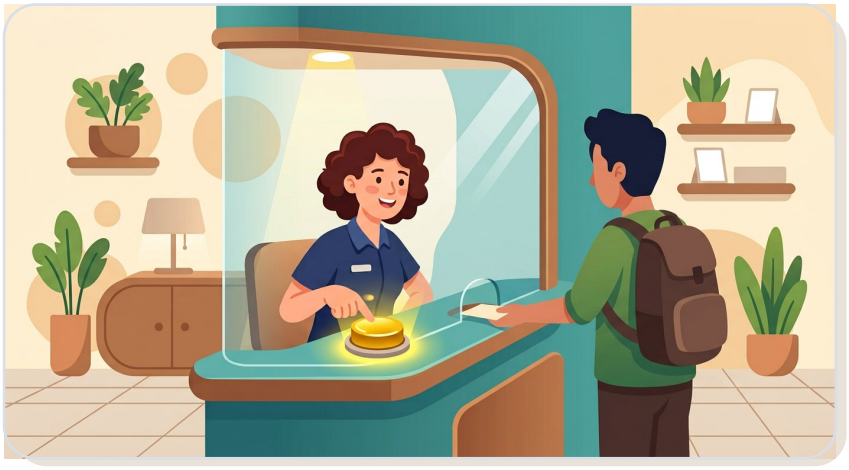
Level 1 (simple): one read-only envelope in memory. A little build-time tool stamps each file with a header (name, size, type) and packs them in a row. The kernel reads them like a roll of tickets.

Level 2 (real): when you need to WRITE files, you meet the **inode** - a library card for each file saying where its pages live. This is how Linux and friends work under the hood.

The librarian: a Virtual File System (VFS)

Should the keyboard and screen be files too? A VFS is one friendly librarian that presents EVERYTHING as folders and files, then quietly hands each “open, read, write” to the right drawer. That is why, on Linux, even your speakers live under /dev!

13 Ringing the service bell: system calls



Apps never walk into the manager's office - they press the service bell.

Visitor-badge apps cannot touch hardware or memory directly. So how do they save a file or print to the screen? They press a **service bell**: a **system call**. It is the one official way for an app to ask the boss for something.

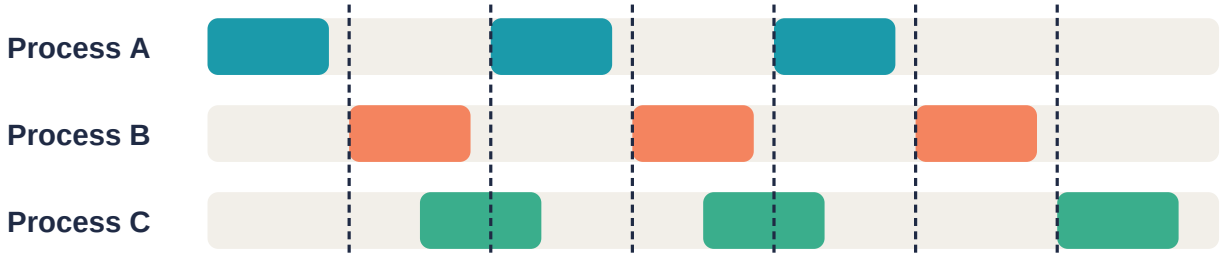
Under the hood, the app fills out a tiny request form (a few numbers saying what it wants), then rings a special doorbell built with a software interrupt. Linux's famous doorbell is number 0x80. The CPU instantly switches into boss uniform, does the job, and hands back the answer - all faster than you can blink.

The reception desk (TSS)

When a visitor rings, the CPU must know WHICH desk to switch to. It consults a tiny structure called the TSS - the reception desk address book - that says: “the boss's desk is over there”. The book sets this up right before entering user mode. Also: the doorbell must be marked “visitors may ring”, or the CPU will refuse the ring!

14 Doing everything at once: multitasking

Your music plays, your download runs, your text editor waits - all “at the same time”. But the CPU juggles only ONE thing at any instant. The trick is switching so fast that it LOOKS simultaneous - like a chef flipping between three pans.



The little clock rings - and the next one gets a turn

So fast that it LOOKS like they all work at the same time

There are two ways to switch. **Polite way (cooperative):** each app, when it can spare a moment, says “your turn!” (a yield system call). Simple and easy to debug - but one rude app can hog everything. **Fair way (preemptive):** a tiny oven timer (the PIT) rings every ~20 milliseconds and the boss FORCES a switch. Everyone gets a fair slice of time.

Switching is like pausing a video game: **every single detail** must be saved - all the registers, the exact instruction spot, the whole uniform state - so the next turn resumes mid-thought as if nothing happened. Each process also gets its OWN notepad (its own stack), so nobody scribbles on anyone else's work.

Pro tip from the book

Build the polite version FIRST. Because it is deterministic (always the same steps in the same order), bugs are far easier to catch. Only then add the oven timer. Slow and steady wins the race - the authors built their whole OS in six weeks exactly this way!

15 You made it! The whole journey

That is the entire book - from a dead piece of metal to a little world that can run programs, show text, listen to the keyboard and juggle tasks. Let us re-tell the whole story in one breath:

The journey in one breath

A computer wakes up (BIOS), hands over to a delivery driver (GRUB), who starts YOUR boss program after checking the secret handshake (0xCAFEBABE). The boss paints the light-board screen, opens a diary line, draws zones and permissions on the warehouse floor (segmentation), installs doorbells (interrupts), invents fake addresses (paging), rents out memory rooms (the allocator), unpacks programs, files paperwork (file systems), opens a service counter (system calls) and learns to juggle (multitasking).

The real book says something wise: build in **tiny steps**, and after every step, TEST it. That habit - more than any single line of code - is how six weeks of steady work turns an empty machine into a working operating system.

And if you ever get stuck? The authors point to the friendly OSDev wiki community, the book's own issue tracker on GitHub, and - above all - patience. Every expert was once exactly where you are now.

The end... and the beginning

Now you know what every computer does the instant you press its power button - and roughly how people build one from scratch. If reading this made you curious about coding, that is the perfect sign. The next tiny step is yours to pick!

PART TWO

LEARN TO CODE, FROM ZERO

no experience needed - by the end of this part you can read and write C

16 What programming really is

Time to open the hood! A **program** is just a list of tiny instructions for an extremely obedient helper. It never guesses, never improvises, and never gets the joke. “Put the milk in the bowl. Put the bowl in the sink.” If you say it wrong, the milk goes in the sink - and that is NOT the helper's fault. That is debugging: finding the wrong step.

Programmers write those instructions in a tidy text file (the **source code**). A special translator program then converts the text into the raw number-codes the CPU actually understands (the **machine code**). Your job is never “math genius” - it is clear thinking, careful spelling, and patience.

The three skills you need (really, just three)

1) Read a recipe and re-type it exactly. 2) When it misbehaves, compare what you WROTE with what you MEANT. 3) Look things up (every programmer does this all day long). Programming is a craft like woodworking: you learn it by doing it, badly at first, then better.

The workshop where programmers work looks scary but is simple: it is the **Terminal** - a window where you type short orders. **cd** = walk into a folder. **ls** = list what is here. **make** = build my project. That is 90% of it. You will be fluent in a week.

17 Your first C programs

We will learn **C** - the language this whole course uses. Make a folder, open a text editor (the built-in TextEdit works; VS Code is nicer), and type this into a file called **hello.c**:

```c
#include <stdio.h>

int main(void) {
    int apples = 3;
    apples = apples + 2;
    printf("I have %d apples!\n", apples);
    return 0;
}
```

hello.c - a complete, runnable program

Read it like English: *include* the printing toolbox; *main* is where every program starts; make a box named **apples** holding the number 3; add 2 to it; print the result. In the Terminal:

```
cc hello.c -o hello
./hello
# prints: I have 5 apples!
```

cc = compile (translate) this; -o = output name

The **%d** is a blank slot filled with the number after the comma. Decisions and repetition: **if (apples > 4) { ... } else { ... }** does one thing or the other; **while (apples > 0) { apples--; }** repeats until the test fails. **apples--** means “one less”.

Try these tiny exercises

1) Print your name and age. 2) Count from 1 to 10 with a while loop (print each number). 3) Print “many” if a number is over 100, else “few”. Type them! Typing is the only way this sticks.

18 Functions: recipes inside recipes

Typing the same steps again and again is how mistakes are born. The fix: name a group of steps once, then call it whenever you need it. That named group is a **function**.

```
#include <stdio.h>

int square(int x) {         /* the recipe: take a number... */
    return x * x;           /* ...give back x times x      */
}

void greet(char *name) {    /* gives nothing back - just acts */
    printf("Hello, %s!\n", name);
}

int main(void) {
    greet("Ada");
    int n = square(5);
    printf("5 squared is %d\n", n);   /* 25 */
    return 0;
}
```

two functions: one returns a value, one just acts

The words before the name say what comes back: **int** = a number, **void** = nothing. The words in parentheses are the **arguments** (the ingredients handed in). And **char *name**? That is a pointer - the very next chapter, and the key to everything the OS code does.

About those odd /* ... */ marks

Anything between slash-star and star-slash is a COMMENT - a note for humans that the compiler ignores. Good programmers comment like this: WHY the code does something, not WHAT it obviously does. In assembly files, comments start with a semicolon (;).

19 Memory, pointers and structs

Remember the warehouse of numbered lockers (chapter 6)? Memory is exactly that: millions of tiny boxes, each with an **address**. When you write **int apples = 3;** the compiler rents a box and sticks the label “apples” on it. A **pointer** is simply a box that holds another box's ADDRESS - a note saying “the treasure is in locker 45067”.

```
int apples = 3;
int *p = &apples;    /* p now holds the ADDRESS of apples   */
printf("%d\n", *p);  /* * means "the treasure at that address" -> 3 */
*p = 10;             /* change apples THROUGH the note: apples == 10 */

int scores[4];       /* an ARRAY: 4 number-boxes in a row  */
scores[0] = 99;      /* first box is number 0! */

struct point {       /* a STRUCT: a labeled pocket with mixed fields */
    int x;
    int y;
};
struct point pt;
pt.x = 7;
```

*& = "the address of", * = "the treasure at"*

Strings are just arrays of letters with a hidden zero at the end: **"hi"** is 'h', 'i', 0. And here is why OS programmers love structs: hardware often hands you a pocket of bytes with an EXACT layout - like the multiboot note or a GDT entry. The magic words **__attribute__((packed))** tell the compiler: “no padding, lay the fields out exactly as I wrote them”. The OS code uses it everywhere - now you can read it!

The one sentence to memorize

A variable is a box with a NAME. A pointer is a box holding an ADDRESS. * follows the address to the treasure. & takes a name and gives you its address. That's the whole idea - everything else is practice.

20 Compiler, linker and Make

How does text become a running program? Two workshop machines:

1) The **compiler** (cc / gcc) translates each .c file into an object file (.o) - half-finished machine code. 2) The **linker** (ld) glues all the .o files together into one finished program, like snapping LEGO sets into a castle. (A third tiny tool, the assembler, does the same for assembly files.)

```
OBJECTS = loader.o kmain.o

kernel.elf: $(OBJECTS)
    ld -T link.ld -melf_i386 $(OBJECTS) -o kernel.elf

%.o: %.c
    gcc -m32 -ffreestanding -fno-builtin -c $< -o $@
```

Makefile excerpt: 'make' reads these recipes and runs them

A **Makefile** is the recipe card for your whole project: what depends on what, and how to rebuild only what changed. You will just type **make** and **make run** - but now you can read what happens underneath.

The strange flags, translated

-m32: speak in 32-bit x86 (our CPU's dialect). **-ffreestanding**: there is NO operating system behind this code - it IS the operating system. **-fno-builtin**: don't assume any library exists. **-nostdlib -nostartfiles**: bring no house furniture - we build our own. **-Wall**: warn me about every smell. The flags sound scary; the reasons are all just “you are the OS now”.

21 Assembly in one sitting

Assembly is the CPU's own language - one line per single machine action. You only need to READ a little of it (and the course's assembly files are small and commented). Meet the cast: the CPU has 8 little hands called **registers**: eax, ebx, ecx, edx (general purpose), esp (stack pointer), ebp (frame base), esi, edi.

```
mov eax, 5        ; eax = 5
add eax, 1        ; eax = 6
cmp eax, 10       ; compare... (sets flags)
jg  bigger        ; ..."jump if greater"
push eax          ; park a value on the STACK
pop ebx           ; take it back into ebx
call my_function  ; run this recipe, then come back
ret               ; ..."recipe finished, go back"
jmp loop_start    ; jump with no conditions
```

the eight-move vocabulary of assembly

The **stack** is a pile of plates: last one on, first one off. It is where function calls park their stuff. And when C code calls assembly (or vice versa) they follow the same handshake, called **cdecl**: arguments go onto the stack right-to-left, and the answer comes back in **eax**. That is exactly how our OS will glue its assembly helpers onto its C brain.

```
push dword 3     ; third argument
push dword 2     ; second argument
push dword 1     ; first argument
call sum_of_three
; the answer is now in eax
```

cdecl in action: this is how assembly calls C

Give yourself permission to skim

Nobody memorizes assembly. Even experts keep a reference open. The goal is to RECOGNIZE the moves: this line stores, this line jumps, this line calls a recipe. That is plenty for building an OS.

22 Set up your Mac workshop

Time to set up the real workshop. The book this course follows was written for Ubuntu, but everything works beautifully on a Mac - Apple Silicon or Intel alike. Open Terminal and install the tools (once):

```
# 1) Homebrew - the Mac's toolbox installer (skip if you have it)
/bin/bash -c "$(curl -fsSL \
  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 2) the OS-building toolchain
brew install nasm i386-elf-gcc i386-elf-binutils qemu
```

two commands and your Mac is a kernel factory

Now grab the course project (it came with this guide) and try it:

```
cd os_tutorial
make          # builds kernel.elf + program.bin
make run      # boots YOUR operating system in QEMU!
```

A window opens and TinyOS boots: text appears, and if a guest program was installed, it drops into user mode and echoes your keystrokes. **QEMU** is the pretend computer (an emulator) - the same one you will use all course long. It is fast to start and impossible to break: if your OS crashes, only the pretend machine dies!

Sanity check - what you should see

In the QEMU window: a banner “Welcome to TinyOS 0.1!”, a step-by-step init log, then READY - and either the echo shell or the user-mode hello. In the Terminal: the same lines through the serial logbook. If you see that, your workshop is PERFECT. Problems? The FAQ in os_tutorial/README.md covers the usual suspects.

PART THREE

ILD YOUR OS, MILESTONE BY MILESTO

nine milestones - each one is small, tested and yours

Welcome to the construction site! The finished project (**os_tutorial/** in the course repository) is only about 1,200 lines of code in total - small enough to truly understand. Each milestone below shows you the beating heart of the code, tells you where the file lives, and what you should see when it works.

```
os_tutorial/
|-- loader.s      first code that runs (multiboot handshake + stack)
|-- kmain.c       the boss's first day at work
|-- fb.c          screen driver          serial.c   logbook driver
|-- cpu.s         loads GDT/IDT tables   gdt.c      segments & privilege
|-- isr.s         interrupt stubs        idt.c      interrupt dispatch
|-- pic.c         interrupt controller   keyboard.c keys -> characters
|-- paging.c      fake addresses on      pmm.c      page-frame landlord
|-- syscall.c     int 0x80 service bell  usermode.c starts user apps
|-- switch.s      the iret into user mode
|-- user/         your first guest program
|-- grub/         GRUB files for the optional bootable ISO
|-- tools/        nasm2gas.py (build even without NASM)
+-- tests/        smoke.py (instant headless check: make smoke)
```

the whole map - every milestone below lights up one of these

A golden rule from the original book: after EVERY milestone - no matter how small - build it and run it. Ten minutes of testing now saves a night of hunting later.

23 Milestone 0: the spark

The first code has three jobs: show the secret handshake so the bootloader agrees to start us, set up a stack (C cannot run without one), and call our C function. That is ALL of **loader.s** - read it slowly, you know every word now:

```
; loader.s - the first code that runs (NASM assembly)
global loader
extern kmain

MAGIC_NUMBER  equ 0x1BADB002             ; the bootloader's magic number
ALIGN_MODULES equ 0x00000001             ; page-align loaded modules
MEMINFO       equ 0x00000002             ; give us a memory map
FLAGS         equ ALIGN_MODULES | MEMINFO
CHECKSUM      equ -(MAGIC_NUMBER + FLAGS)

KERNEL_STACK_SIZE equ 16384              ; 16 KB of stack

section .text
align 4
    dd MAGIC_NUMBER                      ; the handshake itself
    dd FLAGS
    dd CHECKSUM                          ; magic + flags + checksum = 0

loader:
    mov esp, kernel_stack + KERNEL_STACK_SIZE
    push ebx                     ; argument 2: the bootloader's info note
    push eax                     ; argument 1: multiboot magic
    call kmain                   ; jump into C code!
.hang:
    cli
    hlt
    jmp .hang

section .bss
align 16
kernel_stack: resb KERNEL_STACK_SIZE
```

loader.s - complete

And the blueprint that says WHERE it lives in memory - **link.ld**. Everything below 1 MB belongs to the BIOS and old machinery, so we live at 1 MB and up:

```
ENTRY(loader)
SECTIONS
{
    . = 0x00100000;              /* load at 1 MB */
    .text ALIGN(0x1000)  : { *(.text) }
    .rodata ALIGN(0x1000): { *(.rodata*) }
    .data ALIGN(0x1000)  : { *(.data) }
    .bss ALIGN(0x1000)   : { *(COMMON) *(.bss) }
    _kernel_end = .;             /* the landlord needs this label */
}
```

link.ld - complete

Build with **make** and boot with **make run**... nothing appears on screen yet (of course - we have no screen driver!). The victory condition is in the QEMU monitor: press **Ctrl-Alt-2**, type **info registers**, and find **EAX=cafebabe**. Your code is running inside a pretend computer. THAT is the spark.

24 Milestone 1: your C code runs

The stack in loader.s means C can now run. **kmain.c** is the boss's first day - a plain C function. Here is a tiny version you can start with (the full version in the project does much more; we will grow it together):

```
/* kmain.c - start with THIS tiny version */
#include "types.h"

void kmain(u32 magic, u32 mbi_addr) {
    /* The screen is 80x25 tiles at memory address 0xB8000.
       Each tile is 2 bytes: letter + colors. Let us wave hello! */
    volatile u16 *fb = (volatile u16 *)0x000B8000;
    const char *msg = "Hi, I am TinyOS!";
    for (u32 i = 0; msg[i]; i++)
        fb[i] = (u16)(msg[i] | 0x0F00);   /* letter + white-on-black */
    for (;;) { }
}
```

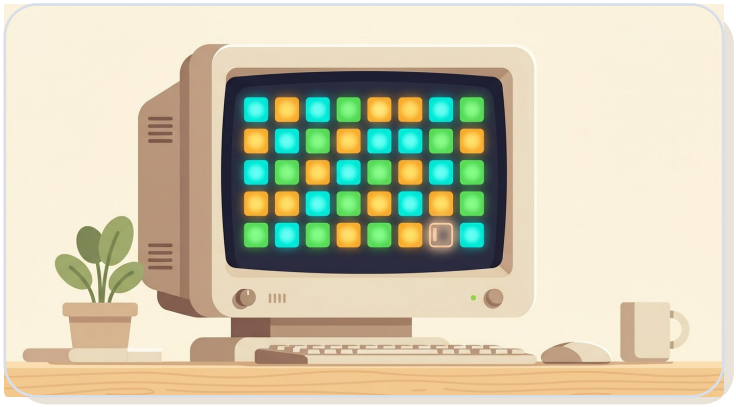
proof of life: the smallest possible C kernel

The cast: **u32** / **u16** are our nicknames for number types (defined in **types.h** - the OS has no standard library, so it brings its own basics). The magic value 0x0F00 = white letters on black. Boot it: words appear! That is C code, running on bare metal, drawing to a real screen.

Milestone 1 victory condition

The QEMU window shows “Hi, I am TinyOS!” on the top-left tile row. If so: delete this toy version and restore the project's kmain.c - from now on we build the real boss.

25 Milestone 2: the screen driver



Recap: the screen is a board of 80x25 light-up tiles.

Poking 0xB8000 raw works, but every part of the OS doing that would be chaos. A **driver** wraps the hardware in a friendly function. Two files: **fb.c** (the screen) and **io.s** (two assembly helpers for the cursor's I/O ports):

```
global outb
global inb

outb:                       ; outb(port, data) - send a byte to a port
    mov al, [esp + 8]       ; the data byte
    mov dx, [esp + 4]       ; the port number
    out dx, al
    ret

inb:                        ; inb(port) - read a byte from a port
    mov dx, [esp + 4]
    xor eax, eax
    in al, dx
    ret
```

io.s - complete (see the cdecl handshake from chapter 21?)

```
void fb_putc(char c) {
    if (c == '\n') { cursor_col = 0; cursor_row++; }
    else {
        fb[cursor_row * FB_COLS + cursor_col] =
            (u16)((u8)c | ((u16)((bg << 4) | fg) << 8));
        cursor_col++;
        if (cursor_col == FB_COLS) { cursor_col = 0; cursor_row++; }
    }
    if (cursor_row == FB_ROWS) scroll();   /* push everything up */
    move_cursor();
}
```

fb.c - the heart of the screen driver (fb.c also scrolls + colors)

See the struct-sized thinking from chapter 19? Each tile packs a letter and two colors into one 16-bit number - exactly the layout the hardware wants. **fb_write(buf, len)**, **fb_clear** and a color setter complete the driver. The whole system now says “print this” and the driver handles the nerdy bits.

26 Milestone 3: the logbook

The serial driver from chapter 5, now real: **serial.c** configures COM1 and writes bytes to it. QEMU shows those bytes in the Terminal (-serial stdio) - a logbook that survives whatever happens on screen:

```
void serial_init(void) {
    serial_configure_baud_rate(1);   /* 115200 bits per second */
    outb(LINE_COMMAND_PORT(COM1), 0x03);  /* 8 bits, no parity, 1 stop */
    outb(FIFO_COMMAND_PORT(COM1), 0xC7);  /* enable + clear the buffers */
    outb(MODEM_COMMAND_PORT(COM1), 0x03); /* ready to talk */
}

void serial_putc(char c) {
    while (!transmit_fifo_empty())   /* wait until the wire is free */
        ;
    outb(DATA_PORT(COM1), (u8)c);
}
```

serial.c - the two functions that matter

Then the finishing touch: **klib.c** with a tiny **kprintf** that prints to BOTH screen and logbook - and a **panic** function that prints the bad news in red and stops. Your future debugging self says thank you.

Milestone 3 victory condition

kmain prints a banner via kprintf - and you see it TWICE: in the QEMU window and in the Terminal. From now on, every milestone's success message rides the logbook.

27 Milestone 4: the boss's uniform

Recap chapter 6 and 8: segments carry privilege levels. Now we build the real GDT with five name tags - null, kernel code/data (boss), user code/data (visitor), plus the **TSS** (the reception desk that tells the CPU which stack to switch to when a visitor rings). **gdt.c** fills a table of 8-byte entries; **cpu.s** loads it:

```
static void gdt_set_gate(int i, u32 base, u32 limit,
                         u8 access, u8 gran) {
    gdt[i].base_low    = (u16)(base & 0xFFFF);
    gdt[i].base_mid    = (u8)((base >> 16) & 0xFF);
    gdt[i].base_high   = (u8)((base >> 24) & 0xFF);
    gdt[i].limit_low   = (u16)(limit & 0xFFFF);
    gdt[i].granularity = (u8)(((limit >> 16) & 0x0F) | (gran & 0xF0));
    gdt[i].access      = access;
}

gdt_set_gate(0, 0, 0, 0, 0);                /* 0x00: the mandatory null */
gdt_set_gate(1, 0, 0xFFFFFFFF, 0x9A, 0xCF); /* 0x08: kernel code, PL0  */
gdt_set_gate(2, 0, 0xFFFFFFFF, 0x92, 0xCF); /* 0x10: kernel data, PL0  */
gdt_set_gate(3, 0, 0xFFFFFFFF, 0xFA, 0xCF); /* 0x18: user code,   PL3  */
gdt_set_gate(4, 0, 0xFFFFFFFF, 0xF2, 0xCF); /* 0x20: user data,   PL3  */
```

gdt.c - the five name tags (0x9A = "code, boss only", 0xFA = "code, visitors too")

```
gdt_flush:                  ; load the GDT, then reload the registers
    mov eax, [esp + 4]
    lgdt [eax]
    mov ax, 0x10            ; kernel DATA selector
    mov ds, ax
    mov es, ax
    mov fs, ax
    mov gs, ax
    mov ss, ax
    jmp 0x08:.flush         ; far jump: reloads CS (code selector)
.flush:
    ret
```

cpu.s - the famous far jump from the segmentation chapter!

The **packed struct** behind each 8-byte entry is exactly why chapter 19 insisted on **__attribute__((packed))**. The bytes must land where the CPU expects them - no padding allowed.

28 Milestone 5: doorbells

Recap chapter 7: interrupts need a phone book (the IDT) and stubs that know which doorbell rang. **isr.s** uses NASM macros - now you can read them - to stamp out 48 nearly-identical stubs:

```
%macro no_error_code_interrupt_handler 1
global isr_%1
isr_%1:
    push dword 0            ; fake error code (this interrupt has none)
    push dword %1           ; which interrupt is this?
    jmp isr_common
%endmacro

isr_common:                 ; the common path for every interrupt
    pusha                   ; save ALL general registers
    mov eax, esp            ; eax points at the saved registers
    push eax                ; ...pushed as the argument to the C function
    call interrupt_handler
    add esp, 4
    popa                    ; restore ALL general registers
    add esp, 8              ; remove error code + interrupt number
    iret                    ; back to whatever was interrupted
```

isr.s - one macro and one common path do the whole job

The C side, **idt.c**, fills the phone book and takes the calls. Look who is inside - the exception names, the keyboard, and the system-call doorbell from chapter 13:

```
void interrupt_handler(cpu_state_t *cpu) {
    if (cpu->int_no < 32) {
        kprintf("EXCEPTION %u (%s) at eip=0x%x\n",
                cpu->int_no, exception_names[cpu->int_no], cpu->eip);
        panic("unhandled CPU exception");
    } else if (cpu->int_no < 48) {
        if (cpu->int_no == 33) keyboard_interrupt();   /* IRQ1 = keyboard */
        pic_acknowledge(cpu->int_no);
    } else if (cpu->int_no == 0x80) {
        cpu->eax = syscall_dispatch(cpu);   /* the answer goes to eax! */
    }
}
```

idt.c - the switchboard of the whole OS

pic.c does the remap dance (the mailroom chapter of the book) so hardware doorbells arrive as interrupts 32-47, and unmasks ONLY the keyboard. One ritual to remember: after handling, say thank you with **pic_acknowledge** or the mailroom goes silent.

29 Milestone 6: your OS reads keys

The keyboard does not send letters - it sends **scan codes** (chapter 7's Morse taps). **keyboard.c** translates taps to characters with a little table, and parks them in a **ring buffer** (a circular queue - the fancy name for a list that wraps around):

```
void keyboard_interrupt(void) {
    u8 sc = inb(KBD_DATA_PORT);   /* read = "got it!" to the keyboard */
    if (sc & 0x80) return;        /* key RELEASE - ignore for now */
    char c = scancode_to_ascii[sc];
    if (!c) return;
    u32 next = (ring_head + 1) % 256;
    if (next == ring_tail) return;    /* ring full: drop the key */
    ring[ring_head] = c;
    ring_head = next;
}
```

keyboard.c - interrupts fill the ring, the kernel drinks from it

Meanwhile **keyboard_getchar()** waits politely: if the ring is empty it says **sti; hlt** - “wake me when a doorbell rings” (the nap we met in chapter 7). Wire it into the echo shell in kmain.c and the magic moment arrives:

Milestone 6 victory condition - a REAL moment

Boot your OS, click the QEMU window, and TYPE. Your keystrokes appear on the screen one by one. Read that again: an interrupt storm from the keyboard hardware flowed through the PIC, the IDT, your driver, and onto the framebuffer - in YOUR operating system. Make yourself a coffee. You earned it.

30 Milestone 7: fake addresses on

Recap chapter 9: paging hands every program a polite lie. Our version is beautifully small - identity mapping (fake address = real address) in four big 4 MB slices, and one slice marked VISITORS WELCOME for the user program. Here is the whole **paging.c** heart:

```
static u32 page_directory[1024] __attribute__((aligned(4096)));

void paging_init(void) {
    for (int i = 0; i < 4; i++) {          /* the four slices */
        u32 flags = PAGE_PRESENT | PAGE_RW | PAGE_4MB;
        if (i == 1) flags |= PAGE_USER;    /* visitors may enter slice 1 */
        page_directory[i] = (u32)(i * 0x00400000) | flags;
    }

    /* PSE on (4 MB pages), point cr3 at the directory, enable paging */
    asm volatile("mov %%cr4, %%eax; or $0x10, %%eax; mov %%eax, %%cr4"
                 ::: "eax", "memory");
    asm volatile("mov %0, %%cr3" :: "r"(page_directory) : "memory");
    asm volatile("mov %%cr0, %%eax; or $0x80000000, %%eax; \
                  mov %%eax, %%cr0" ::: "eax", "memory");
}
```

paging.c - the cake with four slices (0-4MB boss-only, 4-8MB for guests)

The page directory must sit at a **4096-aligned** address - hence the **aligned(4096)** request to the compiler. Those three little assembly blocks are exactly the “enable paging” recipe from chapter 9: set cr4's PSE bit, load cr3, set cr0's PG bit.

Milestone 7 victory condition

The logbook says “paging: fake addresses ON” and the OS keeps running. Want proof it protects? Try reading address 0 from the kernel after paging is on - the CPU fires exception 14 (page fault) and your new exception handler prints the red panic message with the offending address. Turn the bug off again afterwards!

31 Milestone 8: the landlord

Recap chapter 10: somebody must know which 4 KB page frames are free. **pmm.c** keeps one checkbox (bit) per frame - a **bitmap** - and fills it from the bootloader's memory map. The frame the OS itself sits on? Never rent it out!

```
void pmm_init(multiboot_info_t *mbi, u32 kernel_end) {
    /* start by assuming EVERYTHING is taken... */
    for (u32 i = 0; i < BITMAP_BYTES; i++) bitmap[i] = 0xFF;

    /* ...then trust the bootloader's map of usable RAM */
    if (mbi->flags & MULTIBOOT_INFO_MMAP) {
        /* walk the map entries, mark_range_free(usable parts) */
    } else if (mbi->flags & MULTIBOOT_INFO_MEMORY) {
        /* short version: RAM = 1 MB .. 1 MB + mem_upper */
    }

    mark_range_used(0x00000000, kernel_end);  /* OUR rooms! + BIOS */
}

u32 pmm_alloc_frame(void) {          /* ask for one free 4 KB frame */
    for (u32 f = 0; f < frame_count; f++)
        if (!is_used(f)) { mark_used(f); return f * FRAME_SIZE; }
    return 0;                        /* out of memory */
}
```

pmm.c - the landlord with the checklist

Boot and read the logbook: the OS prints the memory map it found and “page frames: N total, M free (4 KB each)”. Your computer's RAM, itemized by YOUR code. Add a test call to **pmm_alloc_frame()** and watch the free count drop by one - the landlord works.

32 Milestone 9: user mode!



Recap: visitors stay outside the castle - with one service bell.

The finish line. Three moves: (1) the CPU drops into visitor clothes with the ONE instruction that can do it - **iret**, dressed up as a fake interrupt-return (chapter 11's recipe, in **switch.s**):

```
enter_user_mode:
    cli
    mov ax, 0x23            ; user DATA selector (0x20) | RPL 3
    mov ds, ax
    mov es, ax
    push dword 0x23         ; ss  - user data selector | RPL 3
    push dword 0x00800000   ; esp - top of the user stack
    pushfd
    pop eax
    or eax, 0x200           ; turn the INTERRUPT flag back ON
    push eax
    push dword 0x1B         ; cs  - user code selector (0x18) | RPL 3
    push dword 0x00400000   ; eip - first instruction of the program
    iret                    ; knock-knock: user mode!
```

switch.s - complete. This is the whole trick of entering user mode.

(2) The guest program lives at 0x400000 with its stack at 0x800000 - both inside our “visitors welcome” slice. It may only act through the service bell - **int 0x80**:

```
static u32 syscall2(u32 n, u32 a, u32 b) {
    u32 ret;
    __asm__ volatile("int $0x80"           /* ring the service bell! */
                     : "=a"(ret)           /* the answer comes in eax */
                     : "a"(n), "b"(a), "c"(b)
                     : "memory");
    return ret;
}

void main(void) {
    print("Hello from USER MODE!\n");
    /* ...echo keys via SYS_GETCHAR / SYS_PUTC, then: */
    syscall2(2, 0, 0);         /* SYS_EXIT -> back to the kernel */
}
```

user/user.c - your first application

(3) The kernel answers the bell in **syscall.c** - and “SYS_EXIT” shows the final trick: returning to the kernel is just pointing iret somewhere else.

Milestone 9 victory condition - THE FINISH LINE

make run. The logbook announces “entering user mode”, and the screen answers: “Hello from USER MODE!” - from a program with NO privileges, ringing doorbells to be heard. You have built an operating system: booting, drivers, interrupts, paging, memory management, protection and system calls. YOU. From nothing.

33 Your finished OS

It is real: **make smoke** even boots it headlessly and checks the banner, the log and every init stage - proof it works on any machine. You can also package it the classic way from the book (a bootable CD image with GRUB):

```
make iso
qemu-system-i386 -cdrom os.iso -serial stdio
```

the 'proper' bootloader path - grub/ has everything you need

The debugging cookbook (pin this to your fridge)

1) READ THE LOGBOOK - your serial log almost always names the crime scene first. 2) Panic prints an address: look it up in the map (nm kernel.elf). 3) Change ONE thing, rebuild, rerun. Small steps - the golden rule. 4) The exception list (chapter 6) tells you WHAT fired. 5) QEMU's monitor (Ctrl-Alt-2) shows registers: info registers. 6) Stuck for an hour? Walk around the block. This works astonishingly often.

Where to go next? The road is long and delightful: load real ELF programs (chapter 11's exercise), build a file system and a shell around it (chapter 12), add a real malloc to the kernel heap (chapter 10), put several processes side by side with cooperative yielding - then the timer, then preemptive scheduling (chapter 14). Then demand paging. Then a filesystem on a real disk. Then... it never really ends, and that is the joy.

The original *Little Book About OS Development* (which this course wraps around) has more depth on every topic; the OSDev wiki has the collective wisdom of everyone who has done this before you. But look at what you have now: not a vague idea of how computers start - a working operating system, and the ability to read and change every line of it.

Well done, kernel developer.

From “I don't know coding” to a booting, typing, user-mode-running operating system on your Mac. The world needs more people who understand the machine at this level - welcome to their ranks. Now go make it do something nobody has ever made a computer do before.